Kiểm chứng tính đúng đắn hệ thống tính toán của chương trình bằng kiểm duyệt mô hình

Nguyễn Thị Loan

Trường Đại học Công nghệ
Luận văn ThS ngành: Công nghệ phần mềm; Mã số: 60 48 10
Cán bộ hướng dẫn khoa học: TS. Đặng Văn Hưng
Năm bảo vệ: 2012

Abstract. Trình bày về cơ sở lý thuyết của kiểm duyệt mô hình (Model checking): khái niệm và ý nghĩa của kiểm duyệt mô hình, quy trình hoạt động của kiểm duyệt mô hình, đặc trưng của kiểm duyệt mô hình, điểm mạnh và điểm yếu của kiểm duyệt dựa trên mô hình sử dụng logic thời gian (Temporal Logic) mô tả các thuộc tính cần kiểm chứng. Nghiên cứu về công cụ Spin, giao diện Xspin, và ngôn ngữ mô hình hóa Promela, máy trạng thái hữu hạn. Tiến hành xây dựng tiến trình đồng hồ, mô hình hóa hệ thống báo động, báo cháy, kết hợp tiến trình đồng hồ với kỹ thuật kiểm duyệt mô hình để kiểm chứng tính đúng đắn của hệ thống đó.

Keywords: Công nghệ phần mềm; Hệ thống tính toán; Kiểm chứng mô hình

Content.

MỞ ĐẦU

1. Đặt vấn đề

Ngày nay chúng ta phụ thuộc rất nhiều vào hệ thống máy tính cả trong sản xuất lẫn đời sống hàng ngày. Các hệ thống này cần phải đảm bảo sự tin cậy và an toàn khi sử dụng. Do đó chúng cần phải được kiểm duyệt kỹ càng ngay từ mô hình của hệ thống để đảm bảo hệ thống hoạt động chính xác tránh gây thiệt hại cả về con người lẫn tiền của.

Kỹ thuật kiểm chứng mô hình đã được sử dụng để kiểm chứng cho các mô hình hệ thống trong thực tế. Các công cụ kiểm chứng đi kèm hiện nay hay dùng như Spin, Kronos, NuSMV,…

2. Nội dung nghiên cứu

Nội dung đề tài nghiên cứu về kỹ thuật kiểm chứng mô hình (Model Checking), dùng công cụ Spin để thực hiện kiểm chứng mô hình hệ thống báo động, báo cháy, sử dụng ngôn ngữ mô hình hóa Promela để mô hình hóa hệ thống báo động, báo cháy, và mô tả các thuộc tính cần kiểm chứng qua Logic thời gian tuyến tính để kiểm chứng tính đúng đắn của hệ thống báo động, báo cháy qua mô hình của nó.

3. Phương pháp nghiên cứu

- Phương pháp thu thập tài liệu.
- Phương pháp phân tích.

4. Cấu trúc luận văn

Chương 1 trình bày về cơ sở lý thuyết của kiểm duyệt mô hình (Model checking).

Chương 2 trình bày về công cụ Spin, giao diện Xspin, và ngôn ngữ mô hình hóa Promela.

Chương 3 xây dựng tiến trình đồng hồ, kết hợp kỹ thuật kiểm duyệt mô hình và tiến trình đồng hồ để kiểm chứng tính đúng đắn của hệ thống báo động, báo cháy

Phần kết luận tóm tắt kết quả đã đạt được, kết luận, những hạn chế cũng như hướng phát triển trong tương lai của đề tài.

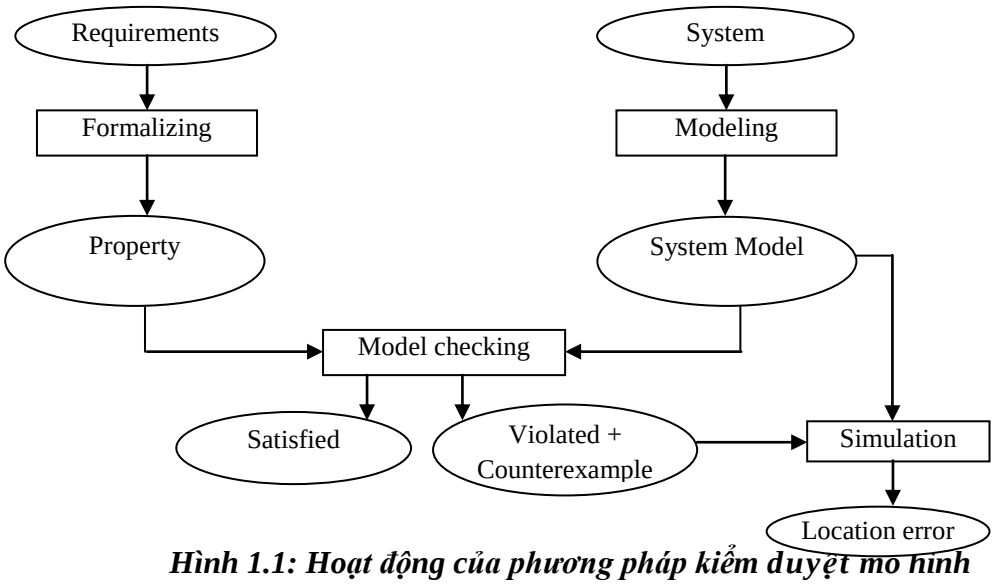
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

1.1. Khái niệm và ý nghĩa của kiểm duyệt mô hình

"Kiểm duyệt mô hình (Model checking) là một kỹ thuật được tự động hóa nhằm đưa ra mô hình hữu hạn trạng thái của hệ thống và thuộc tính hình thức, kỹ thuật này sẽ kiểm tra có hay không thuộc tính được thõa mãn bởi mô hình của hệ thống" [5].

Kiểm duyệt mô hình cho phép kiểm duyệt phần mềm không còn lỗi và thực hiện đúng chức năng đặt ra. Nó kiểm tra mọi khả năng có thể của trạng thái hệ thống.

1.2. Quy trình hoạt động của kiểm duyệt mô hình



Hình 1.1: Hoạt động của phương pháp kiểm duyệt mô hình

Mô hình của hệ thống được xây dựng từ đặc tả của hệ thống. Mô hình này thể hiện hành vi của hệ thống và có thể được viết bởi ngôn ngữ C, Java, hay các ngôn ngữ mô tả phần cứng.

1.3. Đặc trưng của kiểm duyệt mô hình

Quá trình kiểm duyệt một mô hình có thể chia thành những pha như sau:

- Pha mô hình hóa (Modeling).
- Pha thực thi (Running).
- Pha phân tích (Analysis).

1.4. Điểm mạnh và điểm yếu của kiểm duyệt dựa trên mô hình

Kiểm duyệt mô hình có một vài điểm mạnh như [2]:

▪ Là phương pháp kiểm chứng tổng quan được áp dụng cho các ứng dụng trong phạm vi lớn như hệ thống nhúng, công nghệ phần mềm, thiết kế phần cứng,…

▪ Hỗ trợ kiểm duyệt cục bộ, các thuộc tính có thể được kiểm tra riêng lẻ, từ đó tập chung kiểm duyệt các thuộc tính quan trọng trước mà không cần thiết đặc tả hệ thống hoàn chỉnh.

▪ Quá trình kiểm duyệt sau không ảnh hưởng đến các lỗi đã được phát hiện trước đó.

▪ Cung cấp các thông tin có ý nghĩa cho việc gỡ lỗi khi phát hiện một thuộc tính không thỏa mãn.

▪ Kiểm duyệt mô hình có nền tảng của toán học, nó dựa trên lý thuyết thuật toán đồ thị, cấu trúc dữ liệu và logic.

Bên cạnh những ưu điểm trên, phương pháp kiểm duyệt mô hình cũng có những yếu điểm như [2]:

▪ Kiểm duyệt mô hình chủ yếu phù hợp với các ứng dụng điều khiển, không phù hợp với các ứng dụng hướng dữ liệu do khối lượng dữ liệu thường tăng vô hạn.

▪ Kiểm duyệt mô hình kiểm chứng mô hình của hệ thống chứ không phải bản thân hệ thống, mọi kết quả đạt được là về mặt mô hình hệ thống, do đó cần có những kỹ thuật khác hỗ trợ như kiểm duyệt để tìm ra lỗi chế tạo (trong phần cứng) và lỗi lập trình (phần mềm).

1.5. Sử dụng logic thời gian (Temporal Logic) mô tả các thuộc tính cần kiểm chứng

1.5.1. Logic thời gian (Temporal Logic)

Trong các nghiên cứu về kiểm duyệt mô hình, có hai loại logic thời gian hay được xem xét là LTL (Linear Temporal Logic) và CTL (Banching Temporal Logic) [2].

• LTL (Linear Temporal Logic): Logic thời gian tuyến tính. Thời gian có cấu trúc tuyến tính, mỗi trạng thái chỉ có một trạng thái ngay tiếp sau nó.

• CTL (Branching Temporal Logic): Logic thời gian rẽ nhánh. Thời gian có cấu trúc tuyến tính, mỗi trạng thái có nhiều trạng thái ngay tiếp sau nó.

Temporal logic thường được sử dụng để mô tả các thuộc tính cần kiểm chứng.

1.5.2. Các thuộc tính cần kiểm chứng

1.5.2.1. Thuộc tính an toàn (Safety)

Tính an toàn của một chương trình đảm bảo rằng sẽ không bao giờ xảy ra tình huống xấu trong chương trình (*"something bad never happen"*).

1.5.2.2. Thuộc tính sống (Liveness)

Thuộc tính liveness của một chương trình đảm bảo rằng nó có thể thực thi được một chức năng tốt *"good"* nào đó đã đặt ra. *("something good will happen eventually"*).

1.5.2.3. Thuộc tính công bằng (Fairness)

Tính công bằng đảm bảo rằng nếu một sự kiện nào đó ở trạng thái sẵn sàng được thực thi thì đến một lúc nào đó nó sẽ được thực thi.

1.6. Máy trạng thái hữu hạn

1.6.1. Định nghĩa máy trạng thái hữu hạn

Có rất nhiều mô hình được sử dụng trong kiểm chứng phần mềm, trong đó có mô hình máy hữu hạn trạng thái – Finite State Machines (FSM).

Máy hữu hạn trạng thái là một bộ $M = < I, S, O, s_o, \delta, \lambda >$. Trong đó

I: *T*ập các yếu tố đầu vào;

S: *T*ập các trạng thái;

O: Tập thông tin đầu ra;

s_0: Trạng thái ban đầu;

δ: S x I → S là hàm chuyển trạng thái;

λ: S x I → O là hàm thông tin đầu ra.

Mô hình máy hữu hạn trạng thái FSM được sử dụng để mô tả hoạt động của nhiều hệ thống trong thực tế.

1.6.2. Các máy trạng thái hữu hạn trao đổi thông tin

Các máy trạng thái hữu hạn trao đổi thông tin với nhau qua việc truyền các thông báo một cách đồng bộ. Trong quá trình truyền giữa các nhãn cần có sự đồng bộ hóa giữa thông điệp gửi *m* (*!m*) và thông điệp nhận (*?m*). Việc truyền đồng bộ chỉ được thực hiện khi thông điệp gửi và thông điệp nhận là đồng thời và tương ứng [5].

CHƯƠNG 2: GIỚI THIỆU VỀ SPIN VÀ PROMELA

2.1. Ngôn ngữ Promela

Promela là ngôn ngữ mô hình hóa dùng để mô tả hệ thống đồng thời [3]. Chẳng hạn như hệ thống điện thoại, chương trình giao tiếp đa luồng, giao thức mạng,… Nó là ngôn ngữ không tất định, có cú pháp và ngữ nghĩa tương tự ngôn ngữ C [9].

2.1.1. Cấu trúc chương trình Promela

Cấu trúc cơ bản của một trương trình Promela [1]:

- Các khai báo kiểu và khai báo biến;
- Khai báo tiến trình;
- Tiến trình init.

2.1.2. Biến

Cũng như hầu hết các ngôn ngữ lập trình có cấu trúc, Promela yêu cầu các biến phải được khai báo trước khi chúng được sử dụng. Khai báo biến trong Promela giống trong ngôn ngữ lập trình C, bắt đầu là kiểu dữ liệu cơ bản theo sau là một hay nhiều biến được định nghĩa và khởi tạo.

2.1.3. Kiểu dữ liệu

2.1.3.1. Các kiểu dữ liệu cơ bản trong Promela

Các kiểu dữ liệu cơ bản trong Promela được liệt kê trong bảng sau:

Bảng 2.1: Các kiểu dữ liệu cơ bản trong Promela [1]

Type	Size	Range
bit or bool	1	0..1
byte	8	0..255
short	16	$-2^{15}-1..2^{15}-1$
int	32	$-2^{31}-1..2^{31}-1$

2.1.3.2. Kiểu dữ liệu có cấu trúc

- ***Kiểu mảng***
- ***Kiểu cấu trúc (bản ghi)***
- ***Kiểu liệt kê***
- ***Dữ liệu kiểu kênh***

Trong Promela, với dữ liệu kiểu kênh có 2 toán tử ! (gửi) và ? (nhận).

• Trong Promela có hai loại kênh là *Rendezvous channels* và *Buffered channels*.

a) 2.1.3.3. Kiểu mtype

Giả sử ta muốn dùng một kí hiệu cho một số, ta có thể sử dụng *macro define* khai báo tại đầu chương trình – tương tự như ngôn ngữ C:

#define N 10

mtype là một kiểu được sử dụng với chức năng tương tự *define*.

2.1.4. Định danh, hằng, và biểu thức

Định danh có thể là một chữ cái, một ký tự, một dấu chấm hay dấu gạch dưới.

Hằng số là một chuỗi ký tự đại diện cho số nguyên, số thập phân,…

Bảng 2.2: Các toán tử trong Promela [1]

Toán tử	Tên
!	phủ định
++, --	Tăng, giảm
*, /, %	Nhân, chia, modul
+, -	Cộng, trừ
<<, >>	phép dịch trái bit, dịch phải bit
<, <=, >, >=	Toán tử quan hệ
==, !=	Bằng, khác
&	Bitwise và
|	inclusive or
&&	Và
||	hoặc
^	Bitwise xor

(-> :)	biểu thức có điều kiện
=	phép gán
[ ]	chỉ số mảng

2.1.5. Tiến trình

2.1.5.1. Tiến trình process

Một tiến trình *process* được khai báo bắt đầu bởi từ khóa *proctype* và chứa:

- Tên tiến trình.
- Danh sách tham số hình thức.
- Khai báo biến cục bộ.
- Thân tiến trình.

2.1.5.2. Tiến trình init

Bất kỳ một chương trình Promela nào đều phải có một tiến trình *init*, nó giống như hàm *main()* trong C. Việc thực hiện chương trình Promela bắt đầu từ tiến trình *init*.

2.1.5.3. Active proctype

Từ khóa *active* có thể là tiền tố của bất kỳ khai báo tiến trình tiến trình nào.

Tác dụng của *active* là tạo sự kết hợp của tiến trình (được khai báo bởi từ khóa *proctype*) với việc khởi tạo.

Nhiều trường hợp việc khai báo tiến trình sử dụng từ khóa *active* giống như khai báo *proctye* bằng cách dùng mảng có hậu tố tùy chọn theo sau từ khóa *active*.

2.1.6. Run và atomic

2.1.6.1. Run và tiến trình init()

Cách khởi tạo tiến trình trong tiến trình *init* là dùng toán tử *run*, khi tiến trình được khai báo mà không có từ khóa *active*.

2.1.6.2. Atomic

Các lệnh được đặt trong *atomic* { } sẽ được thực hiện như một lệnh độc lập và không bị các lệnh khác ngoài nó chen vào. Việc sử dụng *atomic* nhằm giảm sự phức tạp của mô hình cần kiểm duyệt.

2.1.7. Cấu trúc điều khiển

2.1.7.1. Lệnh lựa chọn if

if

:: biểu_thức_logic1 → Lệnh_11; ...; Lệnh_1n

:: biểu_thức_logic2 → Lệnh_21; ...; Lệnh_2n

...

:: biểu_thức_logicn → Lệnh_n1; ...; Lệnh_nn

fi

Biểu thức logic có thể là *true* hay *false*.

2.1.7.2. Lệnh lặp do

do

::Biểu_thức_logic_1 → Lệnh_11;…; Lệnh_1n;

::Biểu_thức_logic_2 → Lệnh_21;…; Lệnh_2n;

…

::Biểu_thức_logic_n → Lệnh_n1;…; Lệnh_nn;

::Biểu_thức_logic → break;

od;

2.1.7.3. Lệnh nhảy goto

Lệnh *goto* nhảy đến một đoạn chương trình sau một nhãn, tên nhãn được đặt trước dấu hai chấm.

2.2. Công cụ Spin

2.2.1. Kiểm chứng chương trình trong Spin

2.2.1.1. Đặc trưng của SPIN

Spin (Simple Promela Interpreter) là một công cụ kiểm chứng, nó hỗ trỡ ngôn ngữ đặc tả hệ thống Promela. Spin được dùng để theo dõi những lỗi logic ở trong những bản thiết kế của hệ thống phân tán như hệ điều hành, giao thức truyền thông dữ liệu, thuật toán song song, giao thức báo hiệu tàu điện,...Ta có thể sử dụng Spin mà không cần phải dựng lên mô hình dưới dạng đồ thị trạng thái [5].

Spin hỗ trợ kiểm chứng mọi thuộc tính yêu cầu có thể biểu diễn dưới dạng LTL (Linear Temporal Logic – Logic thời gian tuyến tính), hoặc cũng có thể sử dụng các khẳng định – assertion để đặc tả một số thuộc tính cần kiểm chứng.

Ngôn ngữ đặc tả Promela được sử dụng để diễn tả mô hình hệ thống và thuộc tính của nó để kiểm chứng mô hình.

Spin có thể mô phỏng sự thực thi của hệ thống.

Việc sử dụng Spin rất đơn giản, hiệu quả cao, và nó phù hợp để kiểm chứng hệ thống phân tán.

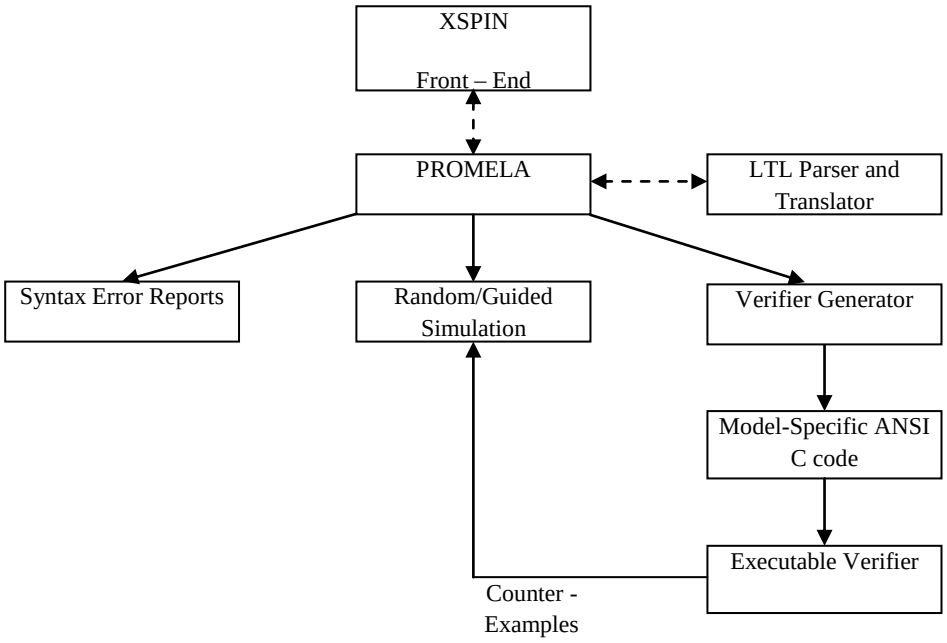
Spin không hỗ trợ kiểm chứng hệ thống vô hạn trạng thái.

b) 2.2.1.2. Mô hình hệ thống trong SPIN

Các hệ thống được mô hình hóa trong Spin như là một tập các tiến trình (mạng các Automata), được chạy song song theo chế độ đan xen và giao tiếp với nhau qua các thông điệp hay qua chia sẻ các biến.

2.2.1.3. Cấu trúc của Spin

Cấu trúc cơ bản của Spin được thể hiện trong hình 2.2.



Hình 2.2: Cấu trúc của Spin [3]

2.2.1.4. Giả lập ngẫu nhiên

Spin biên dịch và chạy một chương trình Promela trong chế độ giả lập ngẫu nhiên rồi in ra các trạng thái của chương trình, sau đó in ra các trạng thái của thuộc tính.

Một trạng thái của chương trình là bộ các giá trị của các biến, biến đếm vị trí các tiến trình. Một tính toán của chương trình là chuỗi các trạng thái bắt đầu bởi trạng thái khởi tạo và tiếp tục với những trạng thái xuất hiện sau khi mỗi câu lệnh được thực hiện [3].

2.2.1.5. Kiểm chứng (Verify)

Assertion là cách đơn giản để kiểm tra chương trình. Một *assertion* là một mệnh đề được đặt trong 1 chương trình mà ta cho rằng mệnh đề sẽ luôn đúng tại vị trí đó. Spin sẽ tính toán các *assertion* trong quá trình tìm kiếm phản ví dụ trong không gian trạng thái của chương trình.

2.2.2. Giao diện người dùng Xspin

Màn hình chính của Xspin là một cửa sổ văn bản hiển thị các tập tin được sử dụng, nó giống như một trình soạn thảo văn bản. Xspin có giao diện thân thiện người dùng cung cấp khả năng chỉnh

2.2.3. Logic thời gian tuyến tính (LTL - Linear Temporal Logic) trong Spin

Khi sử dụng *assertion* bị hạn chế trong việc biểu diễn tính chất của mô hình. Từ đó cần dùng đến logic thời gian tuyến tính LTL, LTL mạnh mẽ hơn trong việc biểu diễn tính chất của mô hình hệ thống.

2.2.3.1. Cú pháp Logic thời gian tuyến tính

Các công thức LTL được tạo ra từ các phép toán mệnh đề, công thức các phép toán mệnh đề được tạo ra từ mệnh đề nguyên thủy (p, q,…) và các phép toán trong bảng 2.3:

Bảng 2.3: Các phép toán mệnh đề trong LTL [6]

Operator	*Math*	*Spin*
not	¬	!
and	∧	&&
or	∨	‖
implies	→	->
equivalent	↔	<->

LTL hình thức hóa các tính chất về thời gian bởi các toán tử thời gian.

Bảng 2.4: Các phép toán thời gian của LTL [6]

Operator	*Math*	*Spin*
always	□	[ ]
eventully	◊	< >
until	*U*	U

Các phép toán □ và ◊ là phép toán một ngôi, phép toán *U* là phép toán hai ngôi. Các phép toán thời gian và phép toán mệnh đề được kết hợp tự do với nhau.

2.2.3.2. Biểu diễn tính chất bất biến của hệ thống bằng LTL

Trong nhiều trường hợp, hệ thống xây dựng phải thỏa mãn một tính chất bất biến nào đó. Với LTL ta biểu diễn tính chất đó bởi toán tử [].

2.2.4. Cấu trúc Never claim

Có những tính chất mà ta mong muốn nó không bao giờ được xuất hiện trong hệ thống. Trong Promela nó được biểu diễn bởi *never claim*.

Trong Spin ta chỉ cần biểu diễn tính chất của hệ thống dưới dạng một biểu thức LTL sau đó đưa vào Spin, Spin sẽ tự động chuyển biểu thức LTL sang cấu trúc *never claim* của Promela dùng vào việc kiểm chứng.

Biểu thức LTL được lưu lại trong tệp *.prp* có tên trùng với tên chương trình. Đoạn mã *never claim* được Spin lưu lại trong tệp *.ltl*.

CHƯƠNG 3: ỨNG DỤNG

3.1. Xây dựng biến và tiến trình đồng hồ

Bàn về kỹ thuật kiểm duyệt mô hình cho hệ thống không có ràng buộc thời gian và có ràng buộc thời gian, có thể đưa ra nhận xét: Với kỹ thuật kiểm duyệt mô hình để kiểm duyệt hệ thống có ràng buộc thời gian thường phức tạp, tốn thời gian nghiên cứu. Ngoài ra nó còn đòi hỏi thời gian chạy lâu và chỉ áp dụng cho những hệ thống nhỏ, bởi hệ thống càng lớn thì số trạng thái càng tăng dẫn tới việc bùng nổ không gian trạng thái. Với kỹ thuật kiểm duyệt mô hình để kiểm duyệt hệ thống không có ràng buộc thời gian dễ tìm hiểu hơn, và hạn chế được phần nào vấn đề bùng nổ không gian trạng thái. Hơn nữa, các kỹ thuật kiểm chứng trợ giúp việc kiểm chứng các tính chất thời gian tổng quát, trong khi các hệ thống có ràng buộc thời gian thực tế thường chỉ đòi hỏi kiểm chứng các tính chất đơn giản ở dạng *deadline*. Từ đó có thể sử dụng kỹ thuật kiểm duyệt mô hình cho hệ thống không có ràng buộc thời gian

kết hợp với kỹ thuật xây dựng biến, tiến trình đồng hồ để kiểm chứng hệ thống có ràng buộc thời gian, cụ thể là hệ thống báo động, báo cháy.

Để có thể kiểm chứng hệ thống thời gian thực cần kết hợp kỹ thuật kiểm duyệt mô hình cho hệ thống không có ràng buộc thời gian với tiến trình đồng hồ. Tiến trình đồng hồ được xây dựng bởi ngôn ngữ mô hình hóa Promela như sau:

```
proctype clock()        {
        int c;          time=-1;
        do
                ::set_time?1->if
                        ::time=-1->time=0;
                        ::else->skip;
                        fi;
                ::set_time?-1->time=-1;
                ::inc?c->if
                        ::time!=-1->time=time+c;
                        :: else -> skip;
                        fi;
        od              }
```

Mục đích xây dựng tiến trình đồng hồ là để đo thời gian khi cần thiết. Hoạt động của nó có thể được mô tả như sau: Ban đầu biến đồng hồ *time = -1*, tức là chưa dùng đồng hồ. Khi đồng hồ bắt đầu được dùng thì biến đồng hồ được thiết lập bằng 0 (*time = 0)*, khi không dùng đồng hồ thì *time = -1*.

Với tiến trình đồng hồ được xây dựng như trên, khi áp dụng vào mô hình hệ thống báo động, báo cháy có ràng buộc thời gian thực là *"hệ thống phải báo động sau 5 giây kể từ khi nhận được thông báo động"*. Ràng buộc này được thể hiện qua biến đồng hồ *time* (*time<5*). Để kiểm chứng ràng buộc thời gian, ta cần sử dụng *assersions* trong Spin: *assert(time<=5)*.

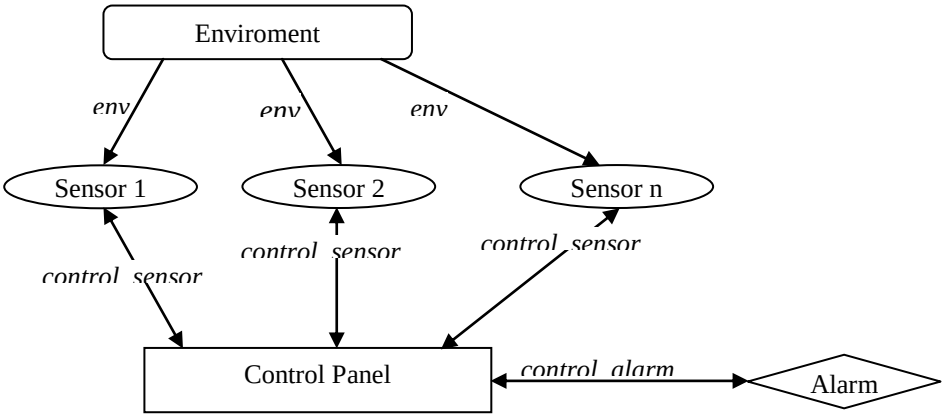
3.2. Hệ thống báo động, báo cháy mức trừu tượng

3.2.1. Mô tả hệ thống mức trừu tượng

Hệ thống báo động bao gồm các *Sensors* để phát hiện động (từ môi trường), bảng điều khiển *Control Panel*, và chuông báo *Alarm*. Khi các *Sensors* nhận được thông báo có động từ môi trường gửi tới, nó sẽ gửi thông báo này cho bảng điều khiển *Control Panel*, ngay sau đó thông báo này cũng sẽ được truyền tới chuông báo *Alarm* từ *Control Panel*. Tại đây *Alarm* sẽ phát âm thanh (chuông báo) để thông báo có động. Hệ thống này không chỉ có mục đích báo động mà còn báo cháy.

Thuộc tính cần kiểm chứng ở đây là bất cứ khi nào có động thì hệ thống phải báo. Để mô tả tính chất này cần sử dụng công thức LTL: *"Bất kỳ khi nào có động -> <> chuông phải báo*".

Kiến trúc trừu tượng của hệ thống báo động, báo cháy:

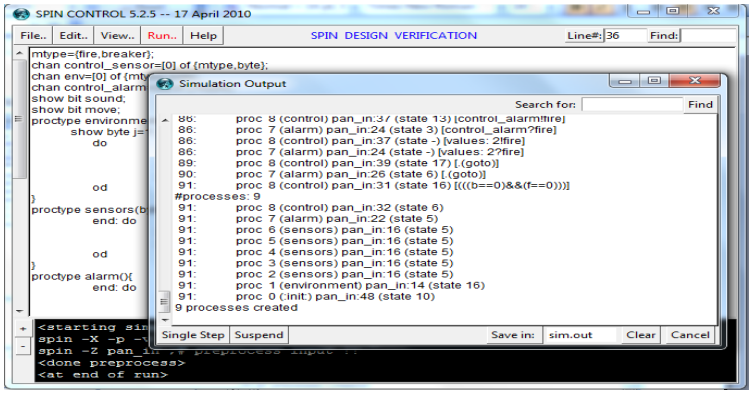


Hình 3.1: Kiến trúc mức 1 của hệ thống báo động, báo cháy

3.2.2. Mô hình Promela cho hệ thống báo động, báo cháy mức trừu tượng và kiểm chứng thuộc tính đơn giản

Mô hình cho hệ thống bằng ngôn ngữ Promela áp dụng cho hệ thống có 5 *Sensors* bao gồm: Các tiến trình *environment(), sensors(i), alarm()*, và *control()*. Các tiến trình giao tiếp với nhau qua các kênh thông điệp. Hệ thống có mục đích báo động, báo cháy tương ứng với hai sự kiện *breaker* (trộm) và *fire* (cháy) .

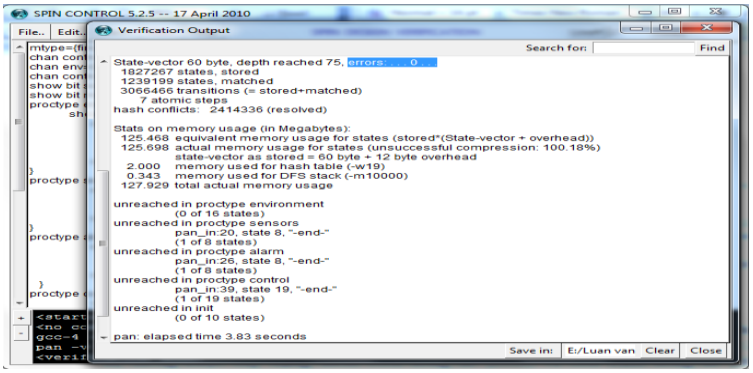
Mô phỏng chương trình trên trong Xspin không có lỗi và được kết quả sau:



Hình 3.2: Kết quả khi mô phỏng mô hình hệ thống báo động, báo cháy mức trừu tượng

Sau khi chạy mô phỏng chương trình như trên, cần kiểm tra tính chất mà hệ thống phải thỏa mãn bằng việc chạy verify. Tính chất cần kiểm chứng của hệ thống báo động, báo cháy mức trừu tượng là: *"bất cứ khi nào có động, hệ thống đều phải báo"*. Biểu thức LTL mô tả tính chất này là: ***[](move-><>sound)***

Spin sẽ tự động sinh ra cấu trúc never claim khi thêm công thức LTL trên. Kết quả chạy verify cho thấy không có lỗi vi phạm do vậy hệ thống thỏa mãn tính chất cần kiểm tra.



Hình 3.3: Kết quả kiểm chứng mô hình hệ thống báo động, báo cháy

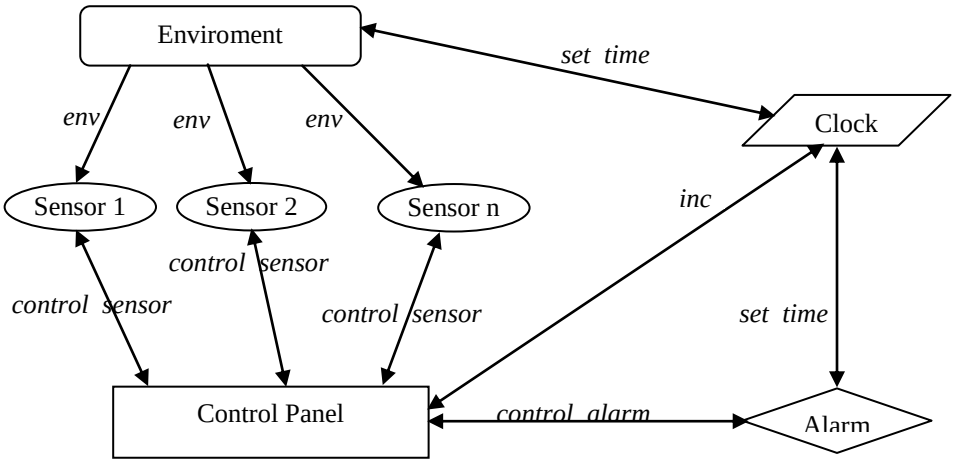
Phân tích kết quả trên thấy errors: 0 nên mô hình đưa ra thỏa mãn thuộc tính đã nêu.

3.3. Hệ thống báo động, báo cháy mức 2

3.3.1. Mô tả hệ thống mức 2

Hệ thống báo động, báo cháy mức 2 là sự mở rộng của mức trừu tượng. Ở mức 2 của hệ thống được xây dựng thêm tiến trình đồng hồ (*clock*). Tiến trình đồng hồ có mục đích là thiết lập thời gian khi cần thiết qua kênh *set_time* và đếm thời gian trôi đi kể từ khi có động cho tới khi chuông phát âm báo động qua biến đồng hồ *time*. Nói cách khác, nhờ xây dựng tiến trình đồng hồ (*clock*) và biến đồng hồ (*time*) ta có thể đưa ràng buộc thời gian thực vào hệ thống báo động, báo cháy qua *assertion*. Hơn nữa tiến trình đồng hồ còn thực hiện tăng thời gian (qua biến đồng hồ *time*) cho mỗi hành động mất thời gian xảy ra, điều này được thực hiện qua kênh *inc*.

Từ mô tả hệ thống như trên ta có thể đưa ra kiến trúc hệ thống như sau:

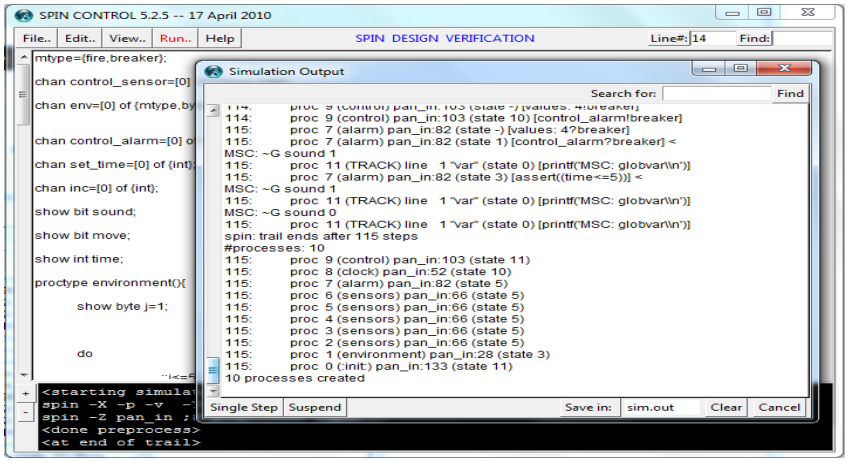


Hình 3.4: Kiến trúc mức 2 của hệ thống báo động, báo cháy

3.3.2. Mô hình Promela cho hệ thống báo động, báo cháy mức 2 và kiểm chứng thuộc tính đơn giản

Từ mô tả hệ thống như trên, ta xây dựng mô hình cho hệ thống bằng ngôn ngữ Promela áp dụng cho hệ thống có 5 *Sensors* bao gồm các biến và tiến trình như ở mức trừu tượng và được bổ sung thêm một vài biến, các thành phần trong các tiến trình ở mức trừu tượng, và tiến trình *clock*.

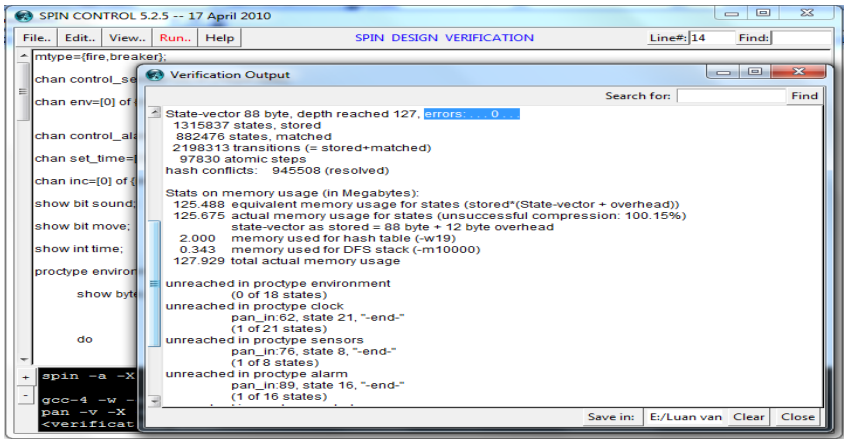
Mô phỏng chương trình trên trong Xspin không có lỗi và được kết quả sau:



Hình 3.5: Kết quả khi mô phỏng mô hình hệ thống báo động, báo cháy mức trừu tượng

Thuộc tính cần kiểm chứng ở đây là: *"bất cứ khi nào có động, hệ thống đều phải báo"* - ***[](move-><>sound)***. Và hệ thống cần đảm bảo *"thời gian từ khi có động đến khi chuông phát báo động không được vượt quá 5 giây"* - *assert(time<=5)*.

Spin sẽ tự động sinh ra cấu trúc *never claim* khi thêm công thức LTL trên. Kết quả chạy verify cho thấy không có lỗi vi phạm do vậy hệ thống thỏa mãn tính chất cần kiểm tra.



Hình 3.6: Kết quả kiểm chứng mô hình hệ thống báo động, báo cháy

Phân tích kết quả trên thấy errors: 0 nên mô hình đưa ra thỏa mãn các thuộc tính đã nêu.

KẾT LUẬN

Với mục tiêu ban đầu của đề tài: "Kiểm chứng tính đúng đắn hệ thống tính toán của chương trình bằng kiểm duyệt mô hình", tác giả đã nghiên cứu về kỹ thuật kiểm duyệt mô hình (Model Checking), tìm hiểu về công cụ kiểm chứng Spin, ngôn ngữ mô hình hóa Promela, xây dựng tiến trình đồng hồ (*clock*), đồng thời đã xây dựng mô hình và kiểm duyệt cho hệ thống báo động, báo cháy, đến nay đã đạt được một số kết quả như sau:

- Nghiên cứu về kỹ thuật kiểm duyệt mô hình (Model Checking).
- Tìm hiểu về công cụ kiểm chứng Spin cũng như giao diện Xspin và ngôn ngữ mô hình hóa Promela.

- Xây dựng biến đồng hồ (*time*) và tiến trình đồng hồ (*clock*) để đo thời gian khi cần thiết, bằng cách này tác giả đã kết hợp tiến trình đồng hồ với kỹ thuật kiểm duyệt mô hình dành cho hệ thống không có ràng buộc thời gian để kiểm chứng hệ thống có ràng buộc thời gian.
- Xây dựng mô hình cho hệ thống báo động, báo cháy bằng ngôn ngữ Promela theo hướng chi tiết hóa dần (incremental).
- Sử dụng kỹ thuật kiểm duyệt mô hình kết hợp tiến trình đồng hồ xây dựng được để kiểm duyệt tính đúng đắn của hệ thống báo động, báo cháy sử dụng công cụ kiểm chứng Spin.
- Quá trình kiểm chứng tác giả đã đưa vào hệ thống 2 thuộc tính cần kiểm chứng là: *"bất cứ khi nào có động, hệ thống đều phải báo"* và "*thời gian từ khi có động tới khi báo động không được vượt quá 5 giây*".

Về cơ bản luận văn đã đạt được những yêu cầu đặt ra. Tuy nhiên trong luận văn còn chưa thực hiện được việc mô hình hóa hệ thống báo động, báo cháy ở mức chi tiết nhất, và chương trình xây dựng được chưa có giao diện. Do đó hướng phát triển trong tương lai của luận văn cần:

- Mô hình hóa hệ thống báo động, báo cháy ở mức chi tiết hơn để hệ thống gần sát với hệ thống thực tế.
- Kiểm duyệt mọi trường hợp có thể của hệ thống để đảm bảo hệ thống hoạt động được đầy đủ chức năng.
- Nghiên cứu phương pháp kiểm tra tính đúng đắn của việc xây dựng mô hình từ mô tả của hệ thống.
- Xây dựng giao diện cho chương trình để thuận tiện khi làm việc với các chức năng.
-

References.

Andrew Ireland, *Distributed Systems Programming F21DS1*, Department of Computer Science School of Mathematical and Computer Sciences Heriot Watt University Edinburgh.

1. Christel Baier Joost - pieter Katoen (2008), *Principles of Model Checking*, MIT Press Cambridge, Massachusetts London, England.
2. Gerard J. Holzmann (2003), *The Spin Model Checker*, Primer and Reference Manual, Addison Wesley Professional.
3. Kim Yong Chun and Dang Van Hung (2004), *Verifying Real – Time Systems Using Untimed Model Checking Tools*, The United Nations University, International Institute for Software Technology.
4. Dang Van Hung, *Model - Checking and the SPIN Modelchecker*, The United Nations University International Institute for Software Technology.
5. Mordechai Ben-Ari (2008), *Principles of the Spin Model Checker*, Springer – Verlag London Limited, London.

6. Rajeev Alur (1991), *Techniques for Automatic Verification of Real Time*, A Dissertation Submitted to the Department of Computer Science and the Committee on Graduate Studies of Stanford University in Partial Fulfillment of the Rquirements for the degree of Doctor of Philosophy.
7. http://spinroot.com.